

Ostad DevOps Assignment 8

The Bastion Host acts as a secure '**jump server**' for administrative access. By placing the internal EC2 instance in a private subnet without an Internet Gateway route, it is shielded from direct internet attacks. The Bastion Host is the only authorized entry point; its security group acts as a filter, ensuring that only users who have successfully authenticated with the Bastion can even attempt to connect to the internal resources.

provider.tf

```
# provider.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region  = "eu-central-1"
  profile = "startup-dev"
}
```

Fig.1. This file configures Terraform to use the **AWS provider** (version 5.0+) to manage resources in the **Frankfurt region** using your "**startup-dev**" local credential profile.

main.tf

Key Point: the SSH key was generated programmatically using the `tls_private_key` resource and stored locally.

```
# Generate an SSH key pair
resource "tls_private_key" "main_key" {
  algorithm = "RSA"
  rsa_bits  = 4096
}

# Store the private key locally
resource "local_file" "private_key_pem" {
  content      = tls_private_key.main_key.private_key_pem
  filename     = "${path.module}/startup_key.pem"
}
```

```

    file_permission = "0400"
}

# Register the public key in AWS
resource "aws_key_pair" "deployer_key" {
    key_name    = "startup-key"
    public_key = tls_private_key.main_key.public_key_openssh
}

# Create Custom VPC
resource "aws_vpc" "main" {
    cidr_block      = "10.0.0.0/16"
    enable_dns_hostnames = true
    tags            = { Name = "StartupVPC" }
}

# Internet Gateway for public access
resource "aws_internet_gateway" "igw" {
    vpc_id = aws_vpc.main.id
    tags   = { Name = "Main-IGW" }
}

# Public Subnet
resource "aws_subnet" "public" {
    vpc_id            = aws_vpc.main.id
    cidr_block        = "10.0.1.0/24"
    map_public_ip_on_launch = true
    availability_zone  = "eu-central-1a"
    tags              = { Name = "PublicSubnet" }
}

# Private Subnet
resource "aws_subnet" "private" {
    vpc_id            = aws_vpc.main.id
    cidr_block        = "10.0.2.0/24"
    availability_zone  = "eu-central-1a"
    tags              = { Name = "PrivateSubnet" }
}

# Public Route Table
resource "aws_route_table" "public_rt" {
    vpc_id = aws_vpc.main.id
    route {
        cidr_block = "0.0.0.0/0"
        gateway_id = aws_internet_gateway.igw.id
    }
}

```

```

}

# Associate Public Route Table
resource "aws_route_table_association" "public_assoc" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public_rt.id
}

# Public Security Group (SSH & HTTP)
resource "aws_security_group" "public_sg" {
  name     = "public_sg"
  vpc_id   = aws_vpc.main.id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # Open SSH for access
  }

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# Private Security Group (SSH only from Public SG)
resource "aws_security_group" "private_sg" {
  name     = "private_sg"
  vpc_id   = aws_vpc.main.id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    security_groups = [aws_security_group.public_sg.id] # Tightly controlled
  }
}

```

```

    egress {
      from_port    = 0
      to_port      = 0
      protocol     = "-1"
      cidr_blocks  = ["0.0.0.0/0"]
    }
  }
}

# Use an Amazon Linux 2 AMI
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}

# Public Instance
resource "aws_instance" "public_ec2" {
  ami                = data.aws_ami.amazon_linux.id
  instance_type      = "t2.micro"
  subnet_id          = aws_subnet.public.id
  key_name            = aws_key_pair.deployer_key.key_name
  vpc_security_group_ids = [aws_security_group.public_sg.id]
  tags               = { Name = "Public-EC2" }
}

# Bastion Host (Assignment lists these as separate, so we make a 2nd public one)
resource "aws_instance" "bastion" {
  ami                = data.aws_ami.amazon_linux.id
  instance_type      = "t2.micro"
  subnet_id          = aws_subnet.public.id
  key_name            = aws_key_pair.deployer_key.key_name
  vpc_security_group_ids = [aws_security_group.public_sg.id]
  tags               = { Name = "Bastion-Host" }
}

# Private Instance
resource "aws_instance" "private_ec2" {
  ami                = data.aws_ami.amazon_linux.id
  instance_type      = "t2.micro"
  subnet_id          = aws_subnet.private.id
  key_name            = aws_key_pair.deployer_key.key_name
  vpc_security_group_ids = [aws_security_group.private_sg.id]
}

```

```

tags                = { Name = "Private-EC2" }
}

# OUTPUTS
output "public_ip" { value = aws_instance.public_ec2.public_ip }
output "bastion_ip" { value = aws_instance.bastion.public_ip }
output "private_internal_ip" { value = aws_instance.private_ec2.private_ip }
output "key_path" { value = local_file.private_key_pem.filename }

```

Fig.2. This code builds a secure network (**VPC**) with an **Internet Gateway**, a **public** subnet containing a **Bastion** host and an **EC2** instance, a **private** subnet with an **isolated EC2**, and the **SSH key pairs** required to access them.

1. Proof of Resource Creation

Plan: 6 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```

+ bastion_ip          = (known after apply)
+ private_internal_ip = (known after apply)
+ public_ip           = (known after apply)

```

aws_subnet.private: Creating...

aws_subnet.public: Creating...

aws_subnet.private: Creation complete after 2s [id=subnet-04450e392ed917307]

aws_instance.private_ec2: Creating...

aws_subnet.public: Still creating... [00m10s elapsed]

aws_instance.private_ec2: Still creating... [00m10s elapsed]

aws_subnet.public: Creation complete after 13s [id=subnet-09fdb5756184f6b41]

aws_route_table_association.public_assoc: Creating...

aws_instance.bastion: Creating...

aws_instance.public_ec2: Creating...

aws_route_table_association.public_assoc: Creation complete after 1s [id=rtbassoc-05f9d2a325c67abd4]

aws_instance.private_ec2: Creation complete after 17s [id=i-0f04f6cb896d8ffa9]

aws_instance.public_ec2: Still creating... [00m10s elapsed]

aws_instance.bastion: Still creating... [00m10s elapsed]

aws_instance.bastion: Creation complete after 16s [id=i-0f2b2991a7e668337]

aws_instance.public_ec2: Creation complete after 16s [id=i-032ef1fa1e4e3c034]

Apply complete! Resources: 6 added, 0 changed, 0 destroyed.

Outputs:

```

bastion_ip = "3.71.184.53"
key_path   = "./startup_key.pem"
private_internal_ip = "10.0.2.80"
public_ip  = "3.71.110.11"

```

PS G:\Ostad\MB>

Fig.3. Resource Creation with **terraform apply** and the output ips

2.

```
PS G:\Ostad\M8> icacls .\startup_key.pem /inheritance:r
processed file: .\startup_key.pem
Successfully processed 1 files; Failed processing 0 files
PS G:\Ostad\M8> icacls .\startup_key.pem /grant:r "$($env:username):(R)"
processed file: .\startup_key.pem
Successfully processed 1 files; Failed processing 0 files
PS G:\Ostad\M8> █
```

Fig.4. These commands secure private keys by **removing inherited permissions** and granting **read-only access** exclusively to my user, preventing SSH from rejecting the key for being too "open."

3.

```
PS G:\Ostad\M8> ssh -i startup_key.pem ec2-user@3.71.184.53
The authenticity of host '3.71.184.53 (3.71.184.53)' can't be established.
ED25519 key fingerprint is SHA256:Gg9mcpLNJtzx0nDcwm+kEeGN3hI/vhrD1eM4uIpDboM.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.71.184.53' (ED25519) to the list of known hosts.

      #_
     ~\  #####_      Amazon Linux 2
    ~\  \#####\
    ~\  \###|      AL2 End of Life is 2026-06-30.
    ~\  \#/
    ~\  V~' '->
    ~\  /
   ~\  /      A newer version of Amazon Linux is available!
  ~\  /
 _/m/'      Amazon Linux 2023, GA and supported until 2028-03-15.
           https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-0-1-240 ~]$ ping 10.0.2.80
PING 10.0.2.80 (10.0.2.80) 56(84) bytes of data.
```

Fig.5. **successful login** to the Bastion Host. the Amazon Linux "bird" logo and prompt

4.

```
[ec2-user@ip-10-0-1-240 ~]$ ping 10.0.2.80
PING 10.0.2.80 (10.0.2.80) 56(84) bytes of data.

^C
PING 10.0.2.80 (10.0.2.80) 56(84) bytes of data.

^C
^C
```

Fig.6. **100% packet loss** is exactly as expected, needs ssh instead

5.

```
[ec2-user@ip-10-0-1-240 ~]$ ssh 10.0.2.80
The authenticity of host '10.0.2.80 (10.0.2.80)' can't be established.
ECDSA key fingerprint is SHA256:od65bI2q+9Wt+zucbMcs3wW/1E0oU7jGX8RLPlXAT3c.
ECDSA key fingerprint is MD5:a7:d6:b6:a8:4e:35:c0:fa:f0:7c:6b:69:76:68:9a:a0.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.0.2.80' (ECDSA) to the list of known hosts.
Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
[ec2-user@ip-10-0-1-240 ~]$ client_loop: send disconnect: Connection reset
PS G:\Ostad\M8>
```

Fig.7. Ssh entry to ip

6. terraform plan

```
data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 2s [id=ami-026992d753d5622bc]

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

# aws_instance.bastion will be created
+ resource "aws_instance" "bastion" {
  + ami                        = "ami-026992d753d5622bc"
  + arn                       = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone          = (known after apply)
  + cpu_core_count             = (known after apply)
  + cpu_threads_per_core       = (known after apply)
  + disable_api_stop           = (known after apply)
  + disable_api_termination    = (known after apply)
  + ebs_optimized              = (known after apply)
  + enable_primary_ipv6        = (known after apply)
  + get_password_data          = false
  + host_id                    = (known after apply)
  + host_resource_group_arn    = (known after apply)
  + iam_instance_profile       = (known after apply)
  + id                         = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle         = (known after apply)
  + instance_state             = (known after apply)
  + instance_type              = "t2.micro"
```

```

+ ipv6_address_count           = (known after apply)
+ ipv6_addresses               = (known after apply)
+ key_name                     = "startup-key"
+ monitoring                   = (known after apply)
+ outpost_arn                  = (known after apply)
+ password_data                = (known after apply)
+ placement_group              = (known after apply)
+ placement_partition_number   = (known after apply)
+ primary_network_interface_id = (known after apply)
+ private_dns                  = (known after apply)
+ private_ip                   = (known after apply)
+ public_dns                   = (known after apply)
+ public_ip                    = (known after apply)
+ secondary_private_ips        = (known after apply)
+ security_groups              = (known after apply)
+ source_dest_check            = true
+ spot_instance_request_id     = (known after apply)
+ subnet_id                    = (known after apply)
+ tags                         = {
    + "Name" = "Bastion-Host"
  }
+ tags_all                     = {
    + "Name" = "Bastion-Host"
  }
+ tenancy                      = (known after apply)
+ user_data                    = (known after apply)
+ user_data_base64            = (known after apply)
+ user_data_replace_on_change = false
+ vpc_security_group_ids       = (known after apply)

+ capacity_reservation_specification (known after apply)

+ cpu_options (known after apply)

+ ebs_block_device (known after apply)

+ enclave_options (known after apply)

+ ephemeral_block_device (known after apply)

+ instance_market_options (known after apply)

+ maintenance_options (known after apply)

+ metadata_options (known after apply)

```



```

+ network_interface (known after apply)

+ private_dns_name_options (known after apply)

+ root_block_device (known after apply)
}

# aws_instance.private_ec2 will be created
+ resource "aws_instance" "private_ec2" {
  + ami                                = "ami-026992d753d5622bc"
  + arn                                = (known after apply)
  + associate_public_ip_address       = (known after apply)
  + availability_zone                 = (known after apply)
  + cpu_core_count                    = (known after apply)
  + cpu_threads_per_core              = (known after apply)
  + disable_api_stop                  = (known after apply)
  + disable_api_termination           = (known after apply)
  + ebs_optimized                     = (known after apply)
  + enable_primary_ipv6               = (known after apply)
  + get_password_data                 = false
  + host_id                           = (known after apply)
  + host_resource_group_arn           = (known after apply)
  + iam_instance_profile              = (known after apply)
  + id                                = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle                = (known after apply)
  + instance_state                    = (known after apply)
  + instance_type                     = "t2.micro"
  + ipv6_address_count                = (known after apply)
  + ipv6_addresses                    = (known after apply)
  + key_name                           = "startup-key"
  + monitoring                        = (known after apply)
  + outpost_arn                       = (known after apply)
  + password_data                     = (known after apply)
  + placement_group                   = (known after apply)
  + placement_partition_number        = (known after apply)
  + primary_network_interface_id      = (known after apply)
  + private_dns                       = (known after apply)
  + private_ip                        = (known after apply)
  + public_dns                        = (known after apply)
  + public_ip                         = (known after apply)
  + secondary_private_ips             = (known after apply)
  + security_groups                   = (known after apply)
  + source_dest_check                 = true
  + spot_instance_request_id          = (known after apply)
  + subnet_id                         = (known after apply)

```

```

+ tags                                     = {
  + "Name" = "Private-EC2"
}
+ tags_all                               = {
  + "Name" = "Private-EC2"
}
+ tenancy                                = (known after apply)
+ user_data                              = (known after apply)
+ user_data_base64                       = (known after apply)
+ user_data_replace_on_change            = false
+ vpc_security_group_ids                 = (known after apply)

+ capacity_reservation_specification (known after apply)

+ cpu_options (known after apply)

+ ebs_block_device (known after apply)

+ enclave_options (known after apply)

+ ephemeral_block_device (known after apply)

+ instance_market_options (known after apply)

+ maintenance_options (known after apply)

+ metadata_options (known after apply)

+ network_interface (known after apply)

+ private_dns_name_options (known after apply)

+ root_block_device (known after apply)
}

# aws_instance.public_ec2 will be created
+ resource "aws_instance" "public_ec2" {
  + ami                                     = "ami-026992d753d5622bc"
  + arn                                     = (known after apply)
  + associate_public_ip_address           = (known after apply)
  + availability_zone                      = (known after apply)
  + cpu_core_count                        = (known after apply)
  + cpu_threads_per_core                  = (known after apply)
  + disable_api_stop                      = (known after apply)
  + disable_api_termination                = (known after apply)
  + ebs_optimized                         = (known after apply)

```

```

+ enable_primary_ipv6           = (known after apply)
+ get_password_data             = false
+ host_id                       = (known after apply)
+ host_resource_group_arn       = (known after apply)
+ iam_instance_profile          = (known after apply)
+ id                            = (known after apply)
+ instance_initiated_shutdown_behavior = (known after apply)
+ instance_lifecycle            = (known after apply)
+ instance_state                = (known after apply)
+ instance_type                 = "t2.micro"
+ ipv6_address_count            = (known after apply)
+ ipv6_addresses                = (known after apply)
+ key_name                      = "startup-key"
+ monitoring                    = (known after apply)
+ outpost_arn                   = (known after apply)
+ password_data                 = (known after apply)
+ placement_group               = (known after apply)
+ placement_partition_number    = (known after apply)
+ primary_network_interface_id  = (known after apply)
+ private_dns                   = (known after apply)
+ private_ip                    = (known after apply)
+ public_dns                    = (known after apply)
+ public_ip                     = (known after apply)
+ secondary_private_ips         = (known after apply)
+ security_groups               = (known after apply)
+ source_dest_check             = true
+ spot_instance_request_id      = (known after apply)
+ subnet_id                     = (known after apply)
+ tags                          = {
  + "Name" = "Public-EC2"
}
+ tags_all                      = {
  + "Name" = "Public-EC2"
}
+ tenancy                       = (known after apply)
+ user_data                     = (known after apply)
+ user_data_base64              = (known after apply)
+ user_data_replace_on_change   = false
+ vpc_security_group_ids        = (known after apply)

+ capacity_reservation_specification (known after apply)

+ cpu_options (known after apply)

+ ebs_block_device (known after apply)

```

```

+ enclave_options (known after apply)

+ ephemeral_block_device (known after apply)

+ instance_market_options (known after apply)

+ maintenance_options (known after apply)

+ metadata_options (known after apply)

+ network_interface (known after apply)

+ private_dns_name_options (known after apply)

+ root_block_device (known after apply)
}

# aws_internet_gateway.igw will be created
+ resource "aws_internet_gateway" "igw" {
  + arn      = (known after apply)
  + id       = (known after apply)
  + owner_id = (known after apply)
  + tags     = {
    + "Name" = "Main-IGW"
  }
  + tags_all = {
    + "Name" = "Main-IGW"
  }
  + vpc_id   = (known after apply)
}

# aws_key_pair.deployer_key will be created
+ resource "aws_key_pair" "deployer_key" {
  + arn            = (known after apply)
  + fingerprint    = (known after apply)
  + id             = (known after apply)
  + key_name       = "startup-key"
  + key_name_prefix = (known after apply)
  + key_pair_id    = (known after apply)
  + key_type       = (known after apply)
  + public_key     = (known after apply)
  + tags_all       = (known after apply)
}

# aws_route_table.public_rt will be created
+ resource "aws_route_table" "public_rt" {

```

```

+ arn                = (known after apply)
+ id                 = (known after apply)
+ owner_id           = (known after apply)
+ propagating_vgws    = (known after apply)
+ route              = [
  + {
    + cidr_block        = "0.0.0.0/0"
    + gateway_id        = (known after apply)
    # (11 unchanged attributes hidden)
  },
]
+ tags_all           = (known after apply)
+ vpc_id             = (known after apply)
}

# aws_route_table_association.public_assoc will be created
+ resource "aws_route_table_association" "public_assoc" {
  + id                = (known after apply)
  + route_table_id    = (known after apply)
  + subnet_id         = (known after apply)
}

# aws_security_group.private_sg will be created
+ resource "aws_security_group" "private_sg" {
  + arn                = (known after apply)
  + description        = "Managed by Terraform"
  + egress              = [
    + {
      + cidr_blocks      = [
        + "0.0.0.0/0",
      ]
      + from_port         = 0
      + ipv6_cidr_blocks  = []
      + prefix_list_ids   = []
      + protocol          = "-1"
      + security_groups   = []
      + self              = false
      + to_port           = 0
      # (1 unchanged attribute hidden)
    },
  ]
+ id                  = (known after apply)
+ ingress              = [
  + {
    + cidr_blocks        = []
    + from_port          = 22
  }
]

```

```

        + ipv6_cidr_blocks = []
        + prefix_list_ids = []
        + protocol         = "tcp"
        + security_groups  = (known after apply)
        + self              = false
        + to_port           = 22
        # (1 unchanged attribute hidden)
    },
]
+ name                = "private_sg"
+ name_prefix         = (known after apply)
+ owner_id            = (known after apply)
+ revoke_rules_on_delete = false
+ tags_all            = (known after apply)
+ vpc_id              = (known after apply)
}

# aws_security_group.public_sg will be created
+ resource "aws_security_group" "public_sg" {
    + arn                = (known after apply)
    + description        = "Managed by Terraform"
    + egress              = [
        + {
            + cidr_blocks = [
                + "0.0.0.0/0",
            ]
            + from_port    = 0
            + ipv6_cidr_blocks = []
            + prefix_list_ids = []
            + protocol      = "-1"
            + security_groups = []
            + self          = false
            + to_port       = 0
            # (1 unchanged attribute hidden)
        },
    ]
    + id                = (known after apply)
    + ingress            = [
        + {
            + cidr_blocks = [
                + "0.0.0.0/0",
            ]
            + from_port    = 22
            + ipv6_cidr_blocks = []
            + prefix_list_ids = []
            + protocol      = "tcp"

```

```

        + security_groups = []
        + self             = false
        + to_port          = 22
          # (1 unchanged attribute hidden)
      },
    + {
      + cidr_blocks = [
        + "0.0.0.0/0",
      ]
      + from_port    = 80
      + ipv6_cidr_blocks = []
      + prefix_list_ids = []
      + protocol     = "tcp"
      + security_groups = []
      + self          = false
      + to_port       = 80
        # (1 unchanged attribute hidden)
    },
  ]
+ name = "public_sg"
+ name_prefix = (known after apply)
+ owner_id = (known after apply)
+ revoke_rules_on_delete = false
+ tags_all = (known after apply)
+ vpc_id = (known after apply)
}

# aws_subnet.private will be created
+ resource "aws_subnet" "private" {
  + arn = (known after apply)
  + assign_ipv6_address_on_creation = false
  + availability_zone = "us-east-1a"
  + availability_zone_id = (known after apply)
  + cidr_block = "10.0.2.0/24"
  + enable_dns64 = false
  + enable_resource_name_dns_a_record_on_launch = false
  + enable_resource_name_dns_aaaa_record_on_launch = false
  + id = (known after apply)
  + ipv6_cidr_block_association_id = (known after apply)
  + ipv6_native = false
  + map_public_ip_on_launch = false
  + owner_id = (known after apply)
  + private_dns_hostname_type_on_launch = (known after apply)
  + tags = {
    + "Name" = "PrivateSubnet"
  }
}

```

```

+ tags_all                                = {
  + "Name" = "PrivateSubnet"
}
+ vpc_id                                = (known after apply)
}

# aws_subnet.public will be created
+ resource "aws_subnet" "public" {
  + arn                                = (known after apply)
  + assign_ipv6_address_on_creation    = false
  + availability_zone                  = "us-east-1a"
  + availability_zone_id                = (known after apply)
  + cidr_block                         = "10.0.1.0/24"
  + enable_dns64                       = false
  + enable_resource_name_dns_a_record_on_launch = false
  + enable_resource_name_dns_aaaa_record_on_launch = false
  + id                                = (known after apply)
  + ipv6_cidr_block_association_id     = (known after apply)
  + ipv6_native                        = false
  + map_public_ip_on_launch            = true
  + owner_id                           = (known after apply)
  + private_dns_hostname_type_on_launch = (known after apply)
  + tags                                = {
    + "Name" = "PublicSubnet"
  }
  + tags_all                            = {
    + "Name" = "PublicSubnet"
  }
  + vpc_id                              = (known after apply)
}

# aws_vpc.main will be created
+ resource "aws_vpc" "main" {
  + arn                                = (known after apply)
  + cidr_block                         = "10.0.0.0/16"
  + default_network_acl_id             = (known after apply)
  + default_route_table_id             = (known after apply)
  + default_security_group_id          = (known after apply)
  + dhcp_options_id                    = (known after apply)
  + enable_dns_hostnames                = true
  + enable_dns_support                  = true
  + enable_network_address_usage_metrics = (known after apply)
  + id                                = (known after apply)
  + instance_tenancy                    = "default"
  + ipv6_association_id                 = (known after apply)
  + ipv6_cidr_block                     = (known after apply)
}

```



```

+ ipv6_cidr_block_network_border_group = (known after apply)
+ main_route_table_id                  = (known after apply)
+ owner_id                             = (known after apply)
+ tags                                  = {
    + "Name" = "StartupVPC"
  }
+ tags_all                              = {
    + "Name" = "StartupVPC"
  }
}

# local_file.private_key_pem will be created
+ resource "local_file" "private_key_pem" {
  + content                = (sensitive value)
  + content_base64sha256   = (known after apply)
  + content_base64sha512   = (known after apply)
  + content_md5             = (known after apply)
  + content_sha1           = (known after apply)
  + content_sha256         = (known after apply)
  + content_sha512         = (known after apply)
  + directory_permission   = "0777"
  + file_permission        = "0400"
  + filename               = "./startup_key.pem"
  + id                     = (known after apply)
}

# tls_private_key.main_key will be created
+ resource "tls_private_key" "main_key" {
  + algorithm              = "RSA"
  + ecdsa_curve            = "P224"
  + id                    = (known after apply)
  + private_key_openssh    = (sensitive value)
  + private_key_pem        = (sensitive value)
  + private_key_pem_pkcs8  = (sensitive value)
  + public_key_fingerprint_md5 = (known after apply)
  + public_key_fingerprint_sha256 = (known after apply)
  + public_key_openssh     = (known after apply)
  + public_key_pem         = (known after apply)
  + rsa_bits               = 4096
}

```

Plan: 14 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```

+ bastion_ip          = (known after apply)
+ key_path            = "./startup_key.pem"

```

```
+ private_internal_ip = (known after apply)
+ public_ip           = (known after apply)
```

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

```
PS G:\0stad\M8> icacls .\startup_key.pem /inheritance:r
```

```
processed file: .\startup_key.pem
```

```
Successfully processed 1 files; Failed processing 0 files
```

```
PS G:\0stad\M8> icacls .\startup_key.pem /grant:r "$($env:username):(R)"
```

```
processed file: .\startup_key.pem
```

```
Successfully processed 1 files; Failed processing 0 files
```

```
PS G:\0stad\M8>
```

Fig.8. The terraform plan output

8. Destroy

```
aws_security_group.private_sg: Destruction complete after 1s
aws_security_group.public_sg: Destroying... [id=sg-08f2519f2051357d2]
aws_subnet.public: Destruction complete after 1s
aws_subnet.private: Destruction complete after 2s
aws_security_group.public_sg: Destruction complete after 2s
aws_vpc.main: Destroying... [id=vpc-07d8a8d9f12108832]
aws_vpc.main: Destruction complete after 1s
                first-fpc-vpc
>> troy complete! Resources: 14 destroyed.
>> G:\0stad\M8>
```

Fig.9. Resources were destroyed after use.